

Aufgabe 1

1.1

Code

```
SELECT last_name, hire_date
FROM employees
WHERE hire_date >= TO_DATE('01.01.1999', 'DD.MM.YYYY')
AND hire_date < TO_DATE('01.01.2000', 'DD.MM.YYYY');
```

Ausgabe

	LAST_NAME	HIRE_DATE
1	Lorentz	07-FEB-99
2	Mourgos	16-NOV-99
3	Grant	24-MAY-99

1.2

Code

```
SELECT last_name, salary, commission_pct,
ADD_MONTHS(TRUNC(hire_date, 'MM'), 24)
FROM employees
WHERE commission_pct IS NOT NULL;
```

Ausgabe

	LAST_NAME	SALARY	COMMISSION_PCT	ADD_MONTHS (TRUNC
1	Zlotkey	10500	0.2	01-JAN-02
2	Abel	11000	0.3	01-MAY-98
3	Taylor	8600	0.2	01-MAR-00
4	Grant	7000	0.15	01-MAY-01

1.3

Code

```
SELECT last_name, hire_date,
TRUNC(MONTHS_BETWEEN(hire_date, (
SELECT hire_date
FROM employees
WHERE last_name = 'King')) / 12)
AS Jahresunterschied
FROM employees
WHERE department_id = 20 OR department_id = 50
ORDER BY Jahresunterschied DESC;
```

Ausgabe

	LAST_NAME	HIRE_DATE	JAHRESUNTERSCHIED
1	Mourgos	16-NOV-99	12
2	Vargas	09-JUL-98	11
3	Matos	15-MAR-98	10
4	Fay	17-AUG-97	10
5	Davies	29-JAN-97	9
6	Rajs	17-OCT-95	8
7	Hartstein	17-FEB-96	8

1.4

Code

```
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date > (SELECT hire_date
  FROM employees
  WHERE last_name = 'Davies');
```

Ausgabe

	FIRST_NAME	LAST_NAME	HIRE_DATE
1	Diana	Lorentz	07-FEB-99
2	Kevin	Mourgos	16-NOV-99
3	Randall	Matos	15-MAR-98
4	Peter	Vargas	09-JUL-98
5	Eleni	Zlotkey	29-JAN-00
6	Jonathon	Taylor	24-MAR-98
7	Kimberely	Grant	24-MAY-99
8	Pat	Fay	17-AUG-97

1.5

Code

```
SELECT e.first_name, e.last_name, e.salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary AND e.hire_date < m.hire_date;
```

Ausgabe

	FIRST_NAME	LAST_NAME	SALARY
1	Ellen	Abel	11000

Aufgabe 2

2.1

Code

```
SELECT last_name
FROM employees
WHERE last_name LIKE '%e_';
```

Ausgabe

	LAST_NAME
1	Davies
2	Zlotkey
3	Abel
4	Whalen

2.2

Code

```
SELECT last_name
FROM employees
WHERE REGEXP_LIKE(last_name, '^e.*');
```

Ausgabe

	LAST_NAME
1	De Haan

2.3

Code

```
SELECT last_name
FROM employees
WHERE REGEXP_LIKE(last_name, 'a.*a');
```

Ausgabe

	LAST_NAME
1	De Haan
2	Vargas

2.4

Code

```
SELECT employee_id, first_name, last_name,
       UTL_MATCH.JARO_WINKLER_SIMILARITY(UPPER('Margos'), UPPER(last_name)) AS similarity
FROM employees
ORDER BY similarity DESC;
```

Ausgabe

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SIMILARITY
1	124	Kevin	Mourgos	86
2	143	Randall	Matos	85
3	144	Peter	Vargas	77
4	141	Trenna	Rajs	63
5	201	Michael	Hartstein	61
6	104	Bruce	Ernst	57
7	176	Jonathon	Taylor	55
8	142	Curtis	Davies	55
9	205	Shelley	Higgins	53
10	202	Pat	Fay	50
11	174	Ellen	Abel	47
12	100	Steven	King	47
13	103	Alexander	Hunold	44
14	200	Jennifer	Whalen	44
15	149	Eleni	Zlotkey	43
16	107	Diana	Lorentz	43
17	178	Kimberely	Grant	41
18	102	Lex	De Haan	0
19	206	William	Gietz	0
20	101	Neena	Kochhar	0

2.5

Code

```
SELECT employee_id, first_name, last_name,
       UTL_MATCH.JARO_WINKLER_SIMILARITY(UPPER('Irena Mays'),
       UPPER(first_name || ' ' || last_name)) AS similarity
FROM employees
ORDER BY similarity DESC;
```

Ausgabe

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SIMILARITY
1	141	Trenna	Rajs	77
2	202	Pat	Fay	65
3	124	Kevin	Mourgos	63
4	178	Kimberely	Grant	62
5	176	Jonathon	Taylor	61
6	201	Michael	Hartstein	60
7	143	Randall	Matos	60
8	144	Peter	Vargas	58
9	104	Bruce	Ernst	58
10	101	Neena	Kochhar	56
11	200	Jennifer	Whalen	55
12	103	Alexander	Hunold	54
13	100	Steven	King	52
14	174	Ellen	Abel	51
15	102	Lex	De Haan	50
16	149	Eleni	Zlotkey	49
17	107	Diana	Lorentz	49
18	142	Curtis	Davies	49
19	206	William	Gietz	48
20	205	Shelley	Higgins	47

Aufgabe 3

3.1

Code

```
SELECT country_id, COUNT(*)
FROM employees
  JOIN departments USING(department_id)
  JOIN locations USING(location_id)
WHERE salary NOT BETWEEN 5000 AND 12000
GROUP BY country_id;
```

Ausgabe

	COUNTRY_ID	COUNT (*)
1	US	9
2	CA	1

3.2

Code

```
SELECT last_name, job_id, salary
FROM employees
WHERE job_id IN ('SA_REP', 'ST_CLERK')
  AND salary NOT IN (2500, 3500, 7000);
```

Ausgabe

	LAST_NAME	JOB_ID	SALARY
1	Davies	ST_CLERK	3100
2	Matos	ST_CLERK	2600
3	Abel	SA_REP	11000
4	Taylor	SA_REP	8600

3.3

Code

```
SELECT jg.grade_level AS Gehaltsstufe,
  department_name AS Departmentname,
  COUNT(employee_id) AS Mitarbeiteranzahl
FROM employees
  JOIN departments USING(department_id)
  JOIN job_grades jg
  ON salary BETWEEN jg.lowest_sal AND jg.highest_sal
GROUP BY jg.grade_level, department_id, department_name;
```

Ausgabe

	GEHALTSSTUFE	DEPARTMENTNAME	MITARBEITERANZAHL
1	D	Marketing	1
2	C	Marketing	1
3	C	IT	2
4	B	Administration	1
5	C	Sales	1
6	A	Shipping	2
7	E	Executive	3
8	D	Sales	2
9	B	Shipping	3
10	B	IT	1
11	C	Accounting	1
12	D	Accounting	1

3.4

Code

```

SELECT department_name, COUNT(employee_id)
FROM employees
  RIGHT JOIN departments USING(department_id)
-- durch den right-join bleibt die rechte
-- Tabelle vollständig erhalten
-- NULL-Werte in employee_id
-- werden von COUNT nicht berücksichtigt
-- daher ist COUNT dann 0
GROUP BY department_id, department_name
HAVING COUNT(employee_id) < 3;

```

Ausgabe

	DEPARTMENT_NAME	COUNT(EMPLOYEE_ID)
1	Contracting	0
2	Accounting	2
3	Administration	1
4	Marketing	2

3.5

Code

```

SELECT department_name
FROM employees
  RIGHT JOIN departments USING(department_id)
  GROUP BY department_id, department_name
HAVING COUNT(employee_id) = (
  SELECT MAX(COUNT(employee_id))
  FROM employees
  GROUP BY department_id
);

```

Ausgabe

	DEPARTMENT_NAME
1	Shipping

3.6

Code

```

SELECT dep.department_name,
       COALESCE(m.last_name, 'position vacant')
FROM departments dep
     LEFT JOIN employees m -- manager_id kann null sein
ON dep.manager_id = m.employee_id;

```

Ausgabe

	DEPARTMENT_NAME	COALESCE(M.LAST_NAME, 'POSITIONVACANT')
1	Executive	King
2	IT	Hunold
3	Shipping	Mourgos
4	Sales	Zlotkey
5	Administration	Whalen
6	Marketing	Hartstein
7	Accounting	Higgins
8	Contracting	position vacant

3.7

Code

```

SELECT emp.last_name, emp.salary, emp.job_id
FROM employees emp
WHERE salary < 5000
     OR salary < (
    SELECT AVG(salary)
    FROM employees emp_sal
    WHERE emp.job_id = emp_sal.job_id
);

```

Ausgabe

	LAST_NAME	SALARY	JOB_ID
1	Ernst	6000	IT_PROG
2	Lorentz	4200	IT_PROG
3	Rajs	3500	ST_CLERK
4	Davies	3100	ST_CLERK
5	Matos	2600	ST_CLERK
6	Vargas	2500	ST_CLERK
7	Taylor	8600	SA_REP
8	Grant	7000	SA_REP
9	Whalen	4400	AD_ASST

3.8

Code

```

SELECT department_id, department_name, location_id
FROM departments
WHERE department_id NOT IN (
    SELECT department_id
    FROM employees
    WHERE job_id = 'SA_REP'
    AND department_id IS NOT NULL
    -- wichtig weil sonst könnte der Ausdruck unknown sein
);

```

Ausgabe

	DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
1	10	Administration	1700
2	20	Marketing	1800
3	50	Shipping	1500
4	60	IT	1400
5	90	Executive	1700
6	110	Accounting	1700
7	190	Contracting	1700

3.9

Code

```

SELECT department_id
FROM employees
JOIN departments USING(department_id)
WHERE department_id IS NOT NULL
GROUP BY department_id
HAVING COUNT(DISTINCT job_id) = 1;

```

Ausgabe

	DEPARTMENT_ID
1	10
2	60

3.10

Code

```

SELECT last_name, job_id, department_id
FROM employees
WHERE (job_id, department_id) IN (
SELECT job_id, department_id
FROM employees
WHERE employee_id IN (103, 142)
) AND employee_id NOT IN (103, 142);

```

Ausgabe

	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	Ernst	IT_PROG	60
2	Lorentz	IT_PROG	60
3	Rajs	ST_CLERK	50
4	Matos	ST_CLERK	50
5	Vargas	ST_CLERK	50

3.11

Code

```

SELECT last_name || ', ' || job_id AS EmployeeAndJob
FROM employees e1
ORDER BY (
-- Durchschnittsgehalt der Abteilung
SELECT AVG(salary)
FROM employees e2
WHERE e2.department_id = e1.department_id
) DESC NULLS LAST;

```

Ausgabe

	EMPLOYEEANDJOB
1	King, AD_PRES
2	Kochhar, AD_VP
3	De Haan, AD_VP
4	Higgins, AC_MGR
5	Gietz, AC_ACCOUNT
6	Zlotkey, SA_MAN
7	Taylor, SA_REP
8	Abel, SA_REP
9	Fay, MK_REP
10	Hartstein, MK_MAN
11	Lorentz, IT_PROG
12	Ernst, IT_PROG
13	Hunold, IT_PROG
14	Whalen, AD_ASST
15	Davies, ST_CLERK
16	Rajs, ST_CLERK
17	Mourgos, ST_MAN
18	Matos, ST_CLERK
19	Vargas, ST_CLERK
20	Grant, SA_REP